

Smart Dispatch IA

Informe Final - Orquestacion Agentica Ciclica y Memoria Persistente

Recurso	Link
Aplicacion en vivo	https://smart-dispatch-q4xk.onrender.com
Repositorio GitHub	https://github.com/rossanny25/smart-dispatch
Demo Docker local	http://127.0.0.1:8050
Guia de ejecucion	docs/runbook.md
Evidencia de sesion	docs/usage-session-log.md

Nota: la aplicacion publicada usa Render Free. Si la instancia estuvo inactiva, la primera carga puede demorar cerca de 50 segundos o mas mientras el servicio despierta.

Checklist de entrega

Requisito docente	Estado	Evidencia
Aplicacion real funcionando	Cumplido	Demo en Render y Docker local
Links en primera pagina	Cumplido	Tabla inicial
Arquitectura y UML	Cumplido	Diagramas incluidos
Tecnologias justificadas	Cumplido	Tabla tecnica
Capturas y log real	Cumplido	Secciones 9 y 10
UX/UI Nielsen	Cumplido	Seccion 11
Ciberseguridad	Cumplido	Seccion 12
Uso de IA y reflexion LLM/SLM	Cumplido	Secciones 13 y 14

1. Resumen ejecutivo

Smart Dispatch IA es un prototipo educativo de asistencia al despacho tecnico en servicios de campo. El proyecto toma una idea conceptual de medio ciclo, basada en agentes especializados y memoria persistente, y la convierte en una aplicacion real, publicada, ejecutable con Docker y documentada para evaluacion academica.

El sistema ayuda a decidir que tecnico conviene asignar a una orden de trabajo. Para hacerlo, separa el problema en etapas: captura de informacion, analisis de requerimientos, planificacion, evaluacion de restricciones, scoring, confianza y aprendizaje. La aplicacion no reemplaza al despachador: funciona como soporte a la decision y conserva evidencia de por que se recomienda un tecnico.

La evolucion principal frente al trabajo teorico inicial es que la orquestacion deja de ser una descripcion general y pasa a estar formalizada como una maquina de estados deterministica. Las reglas duras se aplican antes del ranking, la funcion objetivo esta versionada, la confianza se calcula separada del puntaje y la memoria persistente queda tratada como evidencia controlada.

2. Contexto y problema

En operaciones de campo, un despachador suele asignar tecnicos con informacion incompleta: tipo de incidente, zona, urgencia, habilidades requeridas, disponibilidad, carga de trabajo, distancia, clima, trafico y experiencia historica. Una mala asignacion puede causar demoras, incumplimiento de SLA, sobrecarga de tecnicos o decisiones poco explicables.

El problema no es solo seleccionar el tecnico mas cercano. Tambien hay restricciones que no deberian negociarse: disponibilidad, certificaciones, turno, jornada maxima, limite de conduccion y elementos de seguridad. Por eso el sistema primero determina factibilidad, luego rankea candidatos y finalmente expone evidencia para que el humano decida.

3. Evolucion desde el trabajo de medio ciclo

Observacion del feedback	Evolucion implementada
Orquestador ambiguo	Maquina de estados deterministica controlada por DispatchOrchestrator.
Memoria persistente poco formalizada	Separacion entre seeds, runtime SQLite y memoria de aprendizaje legacy.
Funcion objetivo descriptiva	Scoring con componentes, pesos, penalizaciones y version de configuracion.
Aprendizaje generico	Aprendizaje incremental/evidencial, sin prometer fine-tuning.
Falta de metricas concretas	KPIs y evidencia requerida documentados para evaluacion futura.
Incertidumbre incompleta	Confianza independiente, warnings y explicaciones estructuradas.
Falta de app real	Render, Docker, screenshots, API evidence y logs reales.

4. Arquitectura general

La arquitectura objetivo es un monolito modular hexagonal con pipeline determinístico. Se eligió monorepo porque el proyecto es académico, el frontend es estático y FastAPI puede servir API y UI desde el mismo proceso. Esto reduce fricción para la evaluación: un repositorio, un README y un comando Docker.

```
Despachador
-> Browser UI
-> FastAPI HTTP Adapter (/api/v1 y /api)
-> Application Commands
-> DispatchOrchestrator
-> Domain Policies (eligibility, scoring, confidence)
-> Unit Of Work / SQLite Repositories
-> SQLite (runs, snapshots, stages, transitions)
```

- frontend/: interfaz HTML, CSS y JavaScript vanilla.
- app/api/v1: API canónica versionada.
- app/application: comandos y casos de uso.
- app/domain: políticas puras de negocio.
- app/adapters: persistencia, legacy API y etapas determinísticas.
- data/seeds: datos reproducibles de demo.
- docs: informe, diagramas, evidencia y runbook.

5. Orquestación agéntica cíclica

El sistema modela el ciclo de despacho como una secuencia de estados. La pieza central es DispatchOrchestrator: los agentes no pueden avanzar estados por su cuenta. Esto responde al feedback docente porque reemplaza el orquestador ambiguo por un mecanismo auditable.

```
[*] -> CAPTURE -> ANALYZE -> PLAN -> EVALUATE
EVALUATE -> WAIT_FOR_DECISION
EVALUATE -> NO_FEASIBLE_CANDIDATES
CAPTURE/ANALYZE/PLAN/EVALUATE -> FAILED
```

Etapas	Responsabilidad
CAPTURE	Normaliza y valida la información de entrada.
ANALYZE	Deriva categoría, prioridad, SLA, certificaciones y duración estimada.
PLAN	Aplica reglas duras y calcula ranking solo para candidatos elegibles.
EVALUATE	Agrega confianza, advertencias y explicación sin cambiar el ranking.
WAIT_FOR_DECISION	Espera la decisión humana del despachador.

6. UML principal

```
WorkOrder 1 -> many DispatchRun
DispatchRun 1 -> many RunSnapshot
DispatchRun 1 -> many StageExecution
DispatchRun 1 -> many StateTransition
DispatchRun 1 -> many EligibilityCandidate
EligibilityCandidate 0..1 -> 0..1 ScoredTechnician
ScoredTechnician 0..1 -> 0..1 ConfidenceOutput
Technician 1 -> many EligibilityCandidate
```

El modelo conserva la separacion entre orden, corrida de despacho, snapshots, ejecuciones por etapa, transiciones de estado, candidatos elegibles, puntajes y salida de confianza. Esta separacion permite explicar cada recomendacion sin mezclar reglas duras, scoring y memoria.

7. Tecnologias usadas

Tecnologia	Uso	Justificacion
Python 3.12	Backend principal	Lenguaje claro y buen ecosistema web/testing.
FastAPI	API HTTP	Contratos claros, OpenAPI y soporte ASGI.
Pydantic v2	Validacion	Modelos estrictos y rechazo de campos desconocidos.
SQLAlchemy Core	Persistencia	SQL explicito sin acoplar dominio a ORM.
Alembic	Migraciones	Control de evolucion del esquema SQLite.
SQLite	Base local	Suficiente para demo academica single-user.
HTML/CSS/JS	Frontend	Interfaz simple, auditable y sin build complejo.
Docker/Compose	Ejecucion	Reproducibilidad local y deploy con Dockerfile.
Render Free	Publicacion	Link publico evaluable.
pytest	Verificacion	Pruebas unitarias, integracion y contrato.
BMad Method	Especificacion	PRD, arquitectura, epicas, historias y trazabilidad.

8. Reglas duras, scoring y memoria

El prototipo distingue entre restricciones duras y criterios de optimización. Un técnico que falla una restricción dura no puede recibir puntaje objetivo. Esta decisión evita que una puntuación alta o una preferencia aprendida oculte una violación operativa.

- Técnico disponible.
- Certificaciones requeridas.
- Turno vigente.
- Jornada máxima.
- Límite de conducción.
- EPP requerido.

$$\text{score} = 0.35 \cdot \text{SLA} + 0.25 \cdot \text{proximidad} + 0.20 \cdot \text{balance_carga} + 0.10 \cdot \text{calidad} + 0.10 \cdot \text{memoria} - \text{penalizaciones}$$

- Técnicos demo: data/seeds/technicians.json.
- Órdenes demo: data/seeds/orders.json.
- Memoria inicial: data/learning_store.json.
- Runtime SQLite local: data/smart_dispatch.db.
- Runtime Docker: volumen smart_dispatch_data.
- Reset académico: POST /api/reset o docker compose down -v.

No se implementó login ni panel de administración porque el alcance de la entrega es un MVP académico de un solo usuario. La carga de información se realiza por seeds JSON versionados y por el flujo de aprendizaje del simulador.

9. Capturas del frontend

The screenshot displays the 'Smart Dispatch IA' dashboard. At the top left, the title 'Smart Dispatch IA' is shown in white, with 'V2.0 • ORQUESTACIÓN CÍCLICA Y MEMORIA PERSISTENTE' below it. To the right, there's a section for 'FACTORES DE ENTORNO:' with a 'Reset' button. Further right, a 'Clima' icon and a 'Solea' button are visible. The main section is titled 'Nueva Solicitud de Servicio' with a clipboard icon. It contains a text area for 'Descripción de la Avería (Lenguaje Natural)' with an example: 'Ej: Urgente fuga de gas en la sucursal Palermo, requiere matriculado para reparar regulador general...'. Below this are two input fields: 'Dirección Física' (containing 'Av. Santa Fe 3400') and 'Zona' (containing 'Palermo'). At the bottom, a blue button with a lightning bolt icon says 'Ingresar y Procesar con IA'.

Smart Dispatch IA
V2.0 • ORQUESTACIÓN CÍCLICA Y MEMORIA PERSISTENTE

FACTORES DE ENTORNO: Reset

Nueva Solicitud de Servicio

Descripción de la Avería (Lenguaje Natural)

Ej: Urgente fuga de gas en la sucursal Palermo, requiere matriculado para reparar regulador general...

Dirección Física: Av. Santa Fe 3400

Zona: Palermo

Ingresar y Procesar con IA

Figura 1. Dashboard inicial con ordenes, tecnicos y controles de contexto.

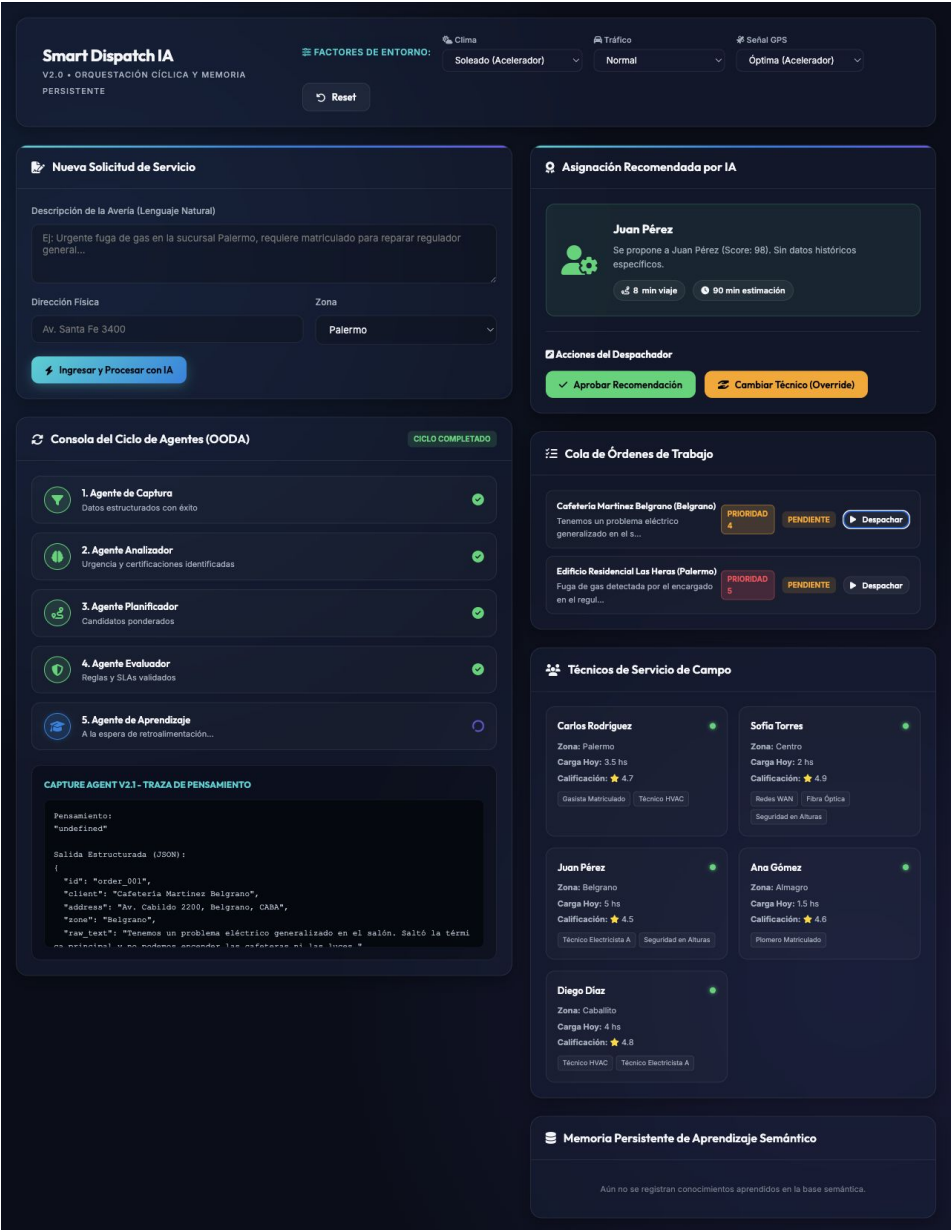


Figura 2. Resultado de simulacion con ciclo agentico y recomendacion.

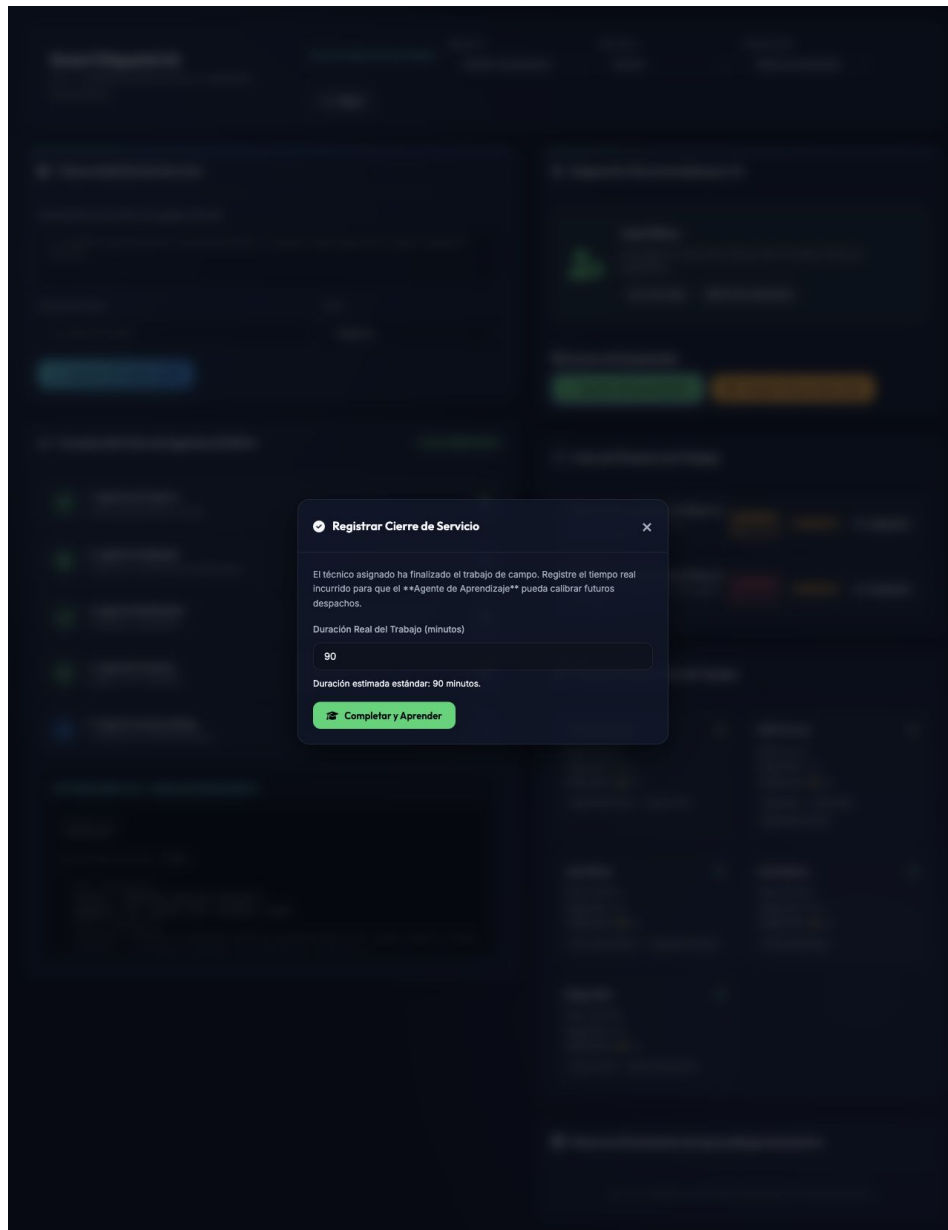


Figura 3. Aprobacion de recomendacion y modal de cierre.

Smart Dispatch IA

V2.0 • ORQUESTACIÓN CÍCLICA Y MEMORIA PERSISTENTE

FACTORES DE ENTORNO:

Clima

Soleado (Acelerador)

Tráfico

Normal

Señal GPS

Óptima (Acelerador)

Reset

Nueva Solicitud de Servicio

Descripción de la Avería (Lenguaje Natural)

Ej: Urgente fuga de gas en la sucursal Palermo, requiere matriculado para reparar regulador general...

Dirección Física

Av. Santa Fe 3400

Zona

Palermo

Ingresar y Procesar con IA

Cola de Órdenes de Trabajo

Cafetería Martínez Belgrano (Belgrano)

Tenemos un problema eléctrico generalizado en el s...

PRIORIDAD 4

COMPLETADA

Edificio Residencial Las Heras (Palermo)

Fuga de gas detectada por el encargado en el regul...

PRIORIDAD 5

PENDIENTE

Despachar

Técnicos de Servicio de Campo

Carlos Rodríguez

Zona: Palermo

Carga Hoy: 3.5 hs

Calificación: ★ 4.7

Gasista Matriculado

Técnico HVAC

Sofía Torres

Zona: Centro

Carga Hoy: 2 hs

Calificación: ★ 4.9

Redes WAN

Fibra Óptica

Seguridad en Alturas

Juan Pérez

Zona: Belgrano

Carga Hoy: 6.5 hs

Calificación: ★ 4.5

Técnico Electricista A

Seguridad en Alturas

Ana Gómez

Zona: Almagro

Carga Hoy: 1.5 hs

Calificación: ★ 4.6

Plomero Matriculado

Diego Díaz

Zona: Caballito

Carga Hoy: 4 hs

Calificación: ★ 4.8

Técnico HVAC

Técnico Electricista A

Memoria Persistente de Aprendizaje Semántico

Aún no se registran conocimientos aprendidos en la base semántica.

Figura 4. Orden completada y aprendizaje registrado.

10. Log de sesion real

Campo	Valor
Fecha	2026-08-11
Runtime	Docker Compose
URL	http://127.0.0.1:8050
Comando	docker compose up --build
Objetivo	Demostrar que la aplicacion existe, corre y sirve frontend/API.

- Se inicio la aplicacion Dockerizada.
- Se abrio el frontend y se verifico /api/technicians.
- Se ejecuto una simulacion de despacho.
- Se aprobo la recomendacion.
- Se completo el servicio y se registro aprendizaje.
- Se exportaron logs Docker/Uvicorn.

Resultado observado: orden Cafeteria Martinez Belgrano, categoria Electricidad, prioridad 4, tecnico recomendado Juan Perez, score visible 98, viaje 8 minutos, duracion estimada 90 minutos y estado final completada.

```
Uvicorn running on http://0.0.0.0:8050
GET / HTTP/1.1 200 OK
GET /api/technicians HTTP/1.1 200 OK
GET /api/orders HTTP/1.1 200 OK
POST /api/dispatch/simulate HTTP/1.1 200 OK
POST /api/dispatch/confirm HTTP/1.1 200 OK
```

11. Autoevaluacion UX/UI con Nielsen

Heuristica	Evaluacion	Mejora
Visibilidad del estado	Buena: etapas y recomendacion visibles.	Mostrar estado canonico DispatchRun.
Relacion con el mundo real	Buena: orden, tecnico, zona y prioridad.	Etiquetar SLA, reglas duras y confianza.
Control del usuario	Media: permite aprobar/cambiar.	Agregar decision canonica completa.
Consistencia	Buena: paneles y estados uniformes.	Unificar errores API en frontend.
Prevencion de errores	Media: backend valida mas que UI.	Validar antes de enviar.
Reconocimiento	Buena: ordenes y tecnicos visibles.	Mantener contexto seleccionado.
Recuperacion de errores	Media: API tiene errores tipados.	Mostrar retry y explicacion no factible.
Ayuda/documentacion	Buena: README, runbook e informe.	Agregar panel breve dentro de la app.

12. Log de ciberseguridad

Riesgo	Mitigacion actual	Pendiente
Exposicion accidental	Default local 127.0.0.1; Docker explicito en 8050.	Auth, HTTPS y politicas de red.
Falta de autenticacion	Demo academica single-user.	Agregar auth antes de uso real.
Datos sensibles	Evidencia demo y recomendacion por zona.	Politica para datos reales.
JSON malformado/grande	/api/v1 limita 1 MiB y usa errores tipados.	Migrar rutas legacy restantes.
Excepciones inseguras	Errores conocidos se mapean a respuestas estables.	Politica productiva completa.
Drift de dependencias	pyproject.toml, uv.lock y Docker pinnean dependencias.	Escaneo de vulnerabilidades.
Migraciones fallidas	Startup fail-closed y backups SQLite.	Retencion/exportacion productiva.
Assets externos	Aceptable en prototipo.	Vendorizacion futura.

13. Uso de IA en co-work

- Interpretar feedback docente y convertirlo en tareas implementables.
- Crear PRD, arquitectura, epicas e historias con BMad.
- Implementar contratos, politicas, persistencia y pruebas.
- Dockerizar la aplicacion.
- Preparar documentacion academica y evidencia.
- Comparar opciones de deploy y publicacion.

La IA tambien tuvo limites: necesito verificacion real para no asumir que dependencias, Docker o SSH funcionaban; la publicacion final dependio de acciones humanas; y las capturas/logs tenian que salir de una ejecucion real, no de una descripcion.

14. Reflexion sobre integracion de LLM o SLM local

La integracion mas razonable de un LLM o SLM local seria como adaptador opcional de ANALYZE. Su funcion seria leer texto libre del incidente y proponer campos estructurados: categoria, prioridad, certificaciones, SLA y duracion estimada.

- No deberia avanzar estados, saltar reglas duras, seleccionar tecnico final, escribir memoria directamente ni inventar evidencia.
- Su salida deberia pasar por contratos Pydantic. Si no valida, el sistema debe rechazarla como salida invalida de etapa.
- Ventajas: privacidad, demo offline, menor dependencia cloud y buen ajuste para Ollama.
- Limitaciones: menor calidad potencial, dependencia de hardware, latencia, pruebas contra alucinaciones y mantenimiento.

15. Despliegue, limitaciones y conclusiones

La aplicacion esta publicada en <https://smart-dispatch-q4xk.onrender.com> y el repositorio esta en <https://github.com/rossanny25/smart-dispatch>. Localmente se ejecuta con `docker compose up --build` y se abre en `http://127.0.0.1:8050`.

- No hay login ni roles.
- No hay panel admin.
- La gestion de datos demo se hace por seeds JSON.
- La UI legacy todavia muestra algunas trazas descriptivas.
- No se implementa aprendizaje semantico completo de produccion.
- No se garantiza persistencia productiva en hosting gratuito.
- No se implementan integraciones reales con GPS, clima o trafico.

Smart Dispatch IA cumple el objetivo central de la entrega final: la idea conceptual evoluciono a una aplicacion real, publicada, ejecutable y documentada. El sistema presenta orquestacion deterministica, reglas duras, scoring, confianza, persistencia, pruebas, Docker, repositorio publico y evidencia de uso.

El aporte principal es mostrar como un sistema agentico puede mantenerse controlado. Los agentes producen evidencia, pero no gobiernan el estado. La memoria puede informar decisiones, pero no reemplaza restricciones de seguridad. La IA puede colaborar en el analisis, pero la aplicacion conserva mecanismos deterministas para que el resultado sea auditable.